TourVault

What changed, why, and how to explain it

Branch	fix/auth-processing-nextjs-web (local only, never pushed)
Committed	f03ce9d: login fix, processing hardening, Next.js + Redux web app
Not committed yet	360 scan points, moving between points, admin API and admin portal
Checks	107 backend tests pass; web app typecheck, lint and production build pass
Date	5 November 2026

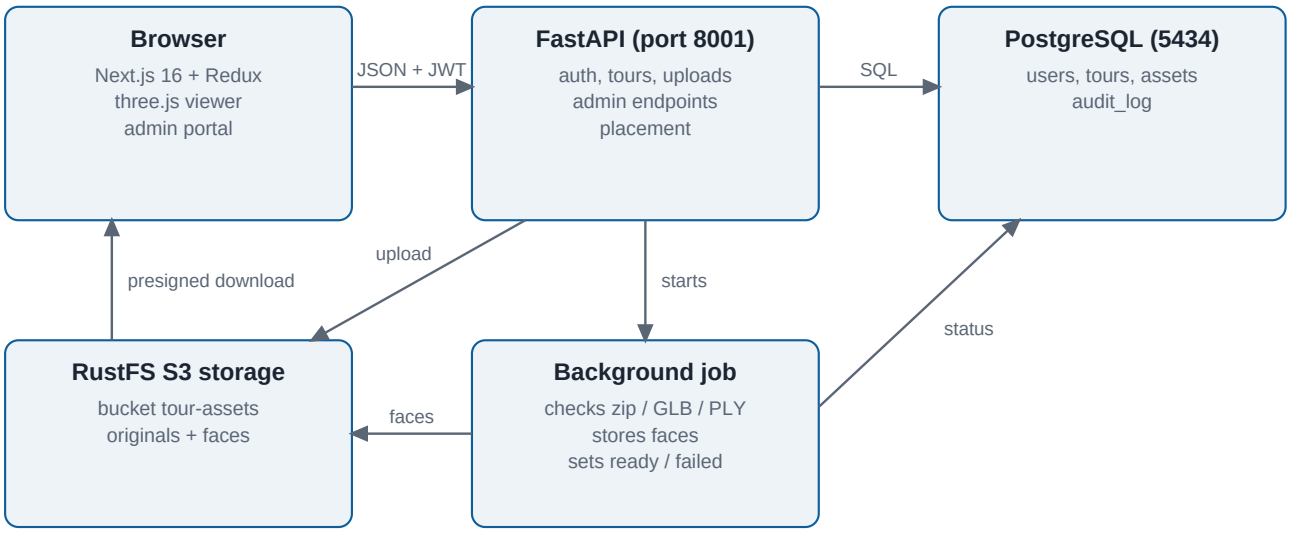
This document walks through every change in the order it was made. Each part says what the problem was, what was changed, where the code lives, and how it was checked. The last section lists questions you are likely to be asked about it, with short answers.

1. Summary

Six pieces of work, in order:

#	Area	In one line	State
1	Login bug	An account registered with an empty password could log in with only its email. Passwords are now required and checked properly.	Committed
2	Processing	Uploads stuck after a restart resume, nothing is processed twice, admins can retry or delete files.	Committed
3	New web app	A Next.js 16 + Redux Toolkit front end replaces the single demo page: login, tours, uploads, 3D viewer.	Committed
4	360 scan points	Upload the six cube images Matterport serves (as Alder Hey's tour does) and look around inside them. No conversion.	Uncommitted
5	Navigation	Scan points have a position and heading; circles on the floor let visitors walk from one to the next.	Uncommitted
6	Admin portal	/admin pages for stats, tours, uploads, a scan point builder, a placement map, users and the activity log.	Uncommitted

How the system fits together



The browser never receives files through the API. It asks the API for short-lived presigned links (10 minutes) and downloads models and cube faces directly from S3 storage. Uploads go through the API so it can check type and size, then a background job processes them.

2. The login bug (committed)

The problem

You reported that entering only an email logged you in. The cause was in registration, not login: the register endpoint accepted an empty password, stored a hash of the empty string, and from then on that account logged in with the email alone because an empty password matched.

What changed

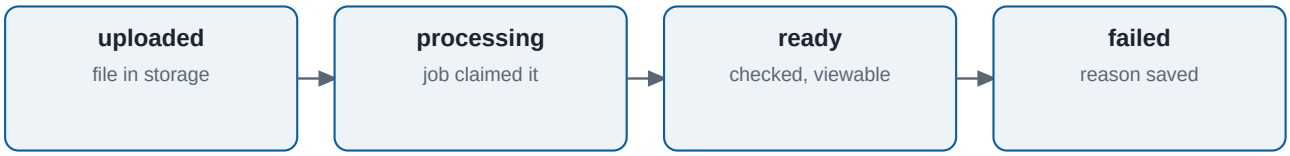
- Passwords must be at least 8 characters and at most 72 bytes (bcrypt ignores anything past 72 bytes, so longer ones were silently cut). Blank or whitespace-only passwords are refused at registration.
- Login refuses an empty password before checking anything else.
- A password over 72 bytes at login is now a normal 401, not a 500 error from bcrypt.
- Emails are compared case-insensitively, so Amal@x.com and amal@x.com are one account.
- Logging in with an unknown email still runs one bcrypt check against a dummy hash. Without it, unknown emails answer faster than known ones, which lets someone find out which emails are registered (a timing attack).
- New `GET /me` returns the logged-in user. The web app calls it on page load to check the saved token is still valid and to get the role from the server.
- New form-based `POST /token` so the Authorize button on the `/docs` page works.

Where

`app/auth.py`, `app/main.py`, `app/models.py`, `app/db.py`; tests in `tests/test_auth.py`.

3. Processing hardening (committed)

Uploaded files go through a lifecycle. The background job moves each one along:



- **Resumed after restart.** Files left in uploaded or processing when the API stopped are picked up again at startup instead of being stuck forever.
- **No double processing.** The job claims a file with one atomic UPDATE (only if it is still in the expected state), so two workers can never process the same file.
- **Retry and delete.** Admins can retry a failed file (`POST /assets/{id}/reprocess`) or delete it (`DELETE /assets/{id}`), which also removes it from storage.
- **Safer uploads.** Empty files are refused, the content type is set by the server rather than trusted from the browser, and a download link is only given once the file is ready.
- **Deleting a tour removes its files from storage too**, not just the database rows.
- GLB files: the binary header's declared length is compared with the real size, which catches truncated uploads. PLY files: the text header is parsed. Only the start of the file is read (HTTP range request).

4. The new web app (committed)

A new `web/` folder holds a Next.js 16 app with Redux Toolkit, Tailwind CSS and three.js. It replaced the old single-file demo page, which is still served at `/app/`.

Part	What it does
authSlice	Token, current user, session status (restoring / authenticated / anonymous).
toursSlice	Tour list, which tour is selected, create / update / delete.
assetsSlice	Files per tour, upload jobs with progress, which file is open in the viewer, placement saves.
adminSlice (new, part 6)	Stats, admin tour list, users and activity for the portal.
Listener middleware	Saves the token to localStorage. Reducers stay pure.
Session restore	On page load the app sends the saved token to `/me`. The server decides if it is valid; the role always comes from the server.
401 handling	Any 401 response ends the session and sends the user to the login page.
Uploads	XMLHttpRequest, because fetch has no upload progress events. While any file is processing, the list polls every 1.5 s.

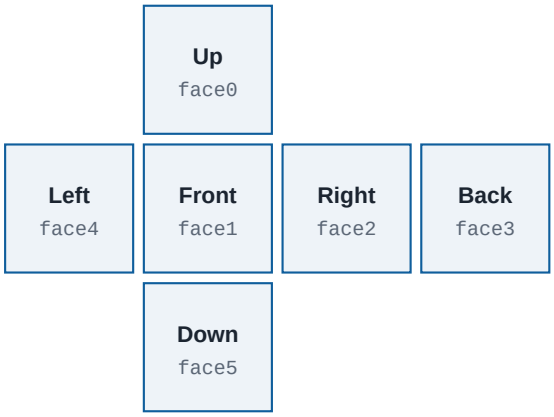
The API allows the web app's origin through `CORS_ORIGINS` (default http://localhost:3000). CI now lints and builds the web app as well as running the tests.

5. 360 scan points (uncommitted)

Why no conversion layer

Alder Hey's public virtual tour is built on Matterport. Matterport does not give you one 3D file; for each place the camera stood (a "sweep" or scan point) it serves six square photos, one for each side of a cube around the camera. You asked to use the same kind of resource rather than converting it into a GLB, so TourVault now accepts exactly that: a zip of six faces, stored and shown unchanged.

The format



Faces can be named the Matterport way (`face0` to `face5`) or by direction (`front`, `back`, `left`, `right`, `up`, `down`). JPG or PNG. An optional `point.json` in the zip places the point (part 6).

```
scan-point.zip
  face0.jpg ... face5.jpg      (or up.jpg, front.jpg, ...)
  point.json  (optional)      {"position": [x, y, z], "heading": 90}
```

What the server checks

In `app/panorama.py`, before anything is stored:

- No unsafe paths in the zip (no folders that climb out with ..), at most 50 entries.
- All six faces present, each one a real JPG or PNG (opened with Pillow, not trusted by extension).
- Every face square and all the same size, at most 8192 pixels a side and 20 MB each.
- point.json, if present, is at most 64 KB, and its numbers are real numbers within a million metres. Heading is normalised to 0 to 360.
- Readable error messages: a failed file says exactly which face is missing or wrong.

What happens after

- The background job stores each face as its own object in S3 and records `kind = panorama`, the face keys and the face size.
- `GET /assets/{id}/download` returns one presigned link per face.
- The viewer draws the six images on the inside of a large cube around the camera, so dragging looks around the room. The new view fades in.

The samples

Three real scan points from Alder Hey's atrium are in `samples/alderhey-local/` with zips that include their Matterport position and heading. That folder is git-ignored: the images are third-party content, so they must not be committed or put on the public EC2 demo.

6. Moving between scan points (uncommitted)

Looking around one point is not a tour. To walk between points, each point needs to know where it is.

Placement

- **position** [x, y, z] in metres, y pointing up.
- **heading** in degrees: how far the cube is turned about the vertical, counter-clockwise seen from above.
- Set it in `point.json` inside the zip, or later with `PUT /assets/{id}/placement` (admin only, 409 if the file is not a scan point).
- Processing keeps a placement that was set by hand: it only fills position and heading when they are empty (SQL COALESCE).

In the viewer

- Every other placed point within 25 m is drawn as a ring on the floor. The floor is 1.5 m below the camera, the eye height Matterport uses.
- Hovering a ring shows "Go to ..."; clicking moves there. A click is a press that moved less than 6 pixels, so dragging to look around never jumps by accident.
- The direction you face is kept when you move, so walking forward feels like walking forward.
- A "Point n of N" bar with previous and next buttons steps through all of a tour's points, placed or not.

The heading formula

For a Matterport sweep, `heading = rotation.y - 180`. This was not guessed: the three Alder Hey points were rendered in TourVault and compared with screenshots from Matterport's own player at the same spots, and clicking point 1's ring lands at the Volunteering desk facing the same way as in Matterport.

7. The admin portal (uncommitted)

Admin API

Endpoint	Returns / does
`GET /admin/stats`	Counts: tours, published tours, files, scan points, 3D models, processing and failed files, storage bytes, users, admins.
`GET /admin/tours`	Every tour with its file count, scan points, models, failed files and total size.
`GET /admin/users`	Users with role, join date and can_log_in (false if saved without a password).
`PUT /admin/users/{id}/role`	Make someone admin or viewer.
`DELETE /admin/users/{id}`	Delete a user.
`GET /admin/activity?limit=`	The audit log, newest first, 1 to 200 entries, with the user's email and tour name.

Safety rules, enforced on the server

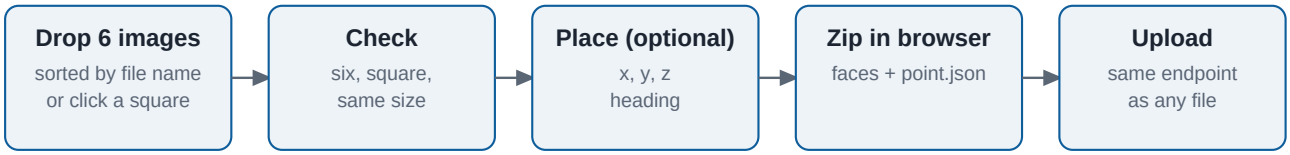
- Every endpoint is admin only. The front end hides the portal from viewers, but that is convenience; the server check is the real protection.
- You cannot change your own role or delete yourself (409), so an admin cannot lock themselves out by mistake.
- The last admin cannot be demoted or deleted (409).
- These checks run under a table lock (LOCK TABLE users IN SHARE ROW EXCLUSIVE MODE). Without it, two admins could demote each other at the same moment, each seeing the other still as admin, and leave none.
- A user with uploads or activity on record cannot be deleted, because the audit log must keep saying who did what. The API says to make them a viewer instead.

Pages

Page	What is on it
/admin Overview	Stat cards, a warning when files failed or are still processing, recent activity, quick links.
/admin/tours	Search, files and size per tour, publish / unpublish, edit, delete with confirm, and a Files button that opens the upload page for that tour.
/admin/upload	Choose a tour, then: the scan point builder, the normal file list (GLB, PLY, zip), and the placement editor with a map.
/admin/users	Role dropdown and delete, with the same rules as the API (your own row and the last admin are disabled).
/admin/activity	The full audit log with a filter by action (created, updated, deleted).

Scan point builder

This is the "upload things like that" part: an admin can build an Alder Hey style scan point from six loose images without making a zip by hand.



- Dropping all six assigns each to the right side from its name. Clicking a square in the cube net sets just that face.
- Position must be all three numbers or none; heading is optional.
- The zip is written in the browser by `web/src/lib/zip.ts`: a small zip writer with CRC-32 and uncompressed (stored) entries. JPG and PNG are already compressed, so compressing again gains nothing.
- The result goes through the normal upload endpoint, so the server checks it exactly like a zip made any other way.
- The zip writer was checked three ways: Python's zipfile test passes, the server's own face and placement readers accept it, and the stored face bytes are identical to the source images. One bug was found and fixed this way (the central directory size was computed too late).

Placement editor

One row per ready scan point with x, y, z and heading, Save and Clear. Underneath, a map seen from above shows every placed point, a tick for the way its front face looks, and a scale bar, so you can see whether points are where they should be.

A Next.js detail

In development Next.js 16 checks that every page can show instantly on navigation. The first version of the admin layout showed a loader instead of the page while the login session was being restored, and Next reported the page as "dropped". The fix: the page is always rendered but hidden until the user is known to be an admin, and each page waits for that before loading its data. The console is now clean on all five pages.

8. Database changes

Five columns on `assets`. Existing databases get them automatically at API startup (`ensure_schema()` with ADD COLUMN IF NOT EXISTS), so nothing has to be run by hand.

Column	Type	Meaning
kind	TEXT	model or panorama
face_keys	TEXT[]	Storage keys of the six faces, in a fixed order
face_size	INTEGER	Pixels per side of each face
position	DOUBLE PRECISION[]	[x, y, z] in metres, or empty
heading	DOUBLE PRECISION	Degrees, 0 to 360, or empty

9. Every API endpoint

Method and path	Who	Purpose
`GET /health`	anyone	Health check
`POST /register`	anyone	Create an account (viewer)
`POST /login, POST /token`	anyone	Get a JWT (30 min); /token is for /docs
`GET /me`	logged in	Current user and role
`GET /tours, GET /tours/{id}`	logged in	List (limit, offset, hospital filter) and read
`POST, PUT, DELETE /tours...`	admin	Create, update (needs version), delete
`POST /tours/{id}/assets`	admin	Upload .glb, .ply or .zip, up to 200 MB
`GET /tours/{id}/assets, GET /assets/{id}`	logged in	List and read files
`GET /assets/{id}/download`	logged in	Presigned link, or six for a scan point
`POST /assets/{id}/reprocess`	admin	Retry a failed file
`PUT /assets/{id}/placement`	admin	New: set position and heading
`DELETE /assets/{id}`	admin	Delete file and storage objects
`GET /legacy/tours`	anyone	Old PHP output, parameterised SQL
`/admin/...`	admin	New: the six admin endpoints in part 7

10. Files changed

Backend

File	Change
`schema.sql`	Five new asset columns
`app/db.py`	Column migration, finish_panorama, set_placement, admin queries, role and delete with table lock
`app/models.py`	Asset fields; AssetPlacement, AdminStats, AdminTour, AdminUser, RoleUpdate, ActivityEntry
`app/panorama.py` (new)	Reads and checks cube faces and point.json
`app/processing.py`	process_panorama: read, store faces, save placement
`app/storage.py`	upload_bytes, read_all
`app/main.py`	.zip uploads, face links, placement endpoint, admin endpoints
`requirements.txt`	pillow 13.0.0 (image checks)
`tests/test_panorama.py` (new)	Zip, face, placement and viewer-data tests
`tests/test_admin.py` (new)	Guards, stats, counts, roles, last admin, delete rules, activity order

Web app

File	Change
`lib/types.ts`	Asset fields, cube faces, admin types
`lib/zip.ts` (new)	CRC-32 and stored zip writer
`store/assetsSlice.ts`	Accepts .zip, face download links, setPlacement
`store/adminSlice.ts` (new)	Portal data and user actions
`components/ModelViewer.tsx`	Rewritten: cube panoramas, rings, hover label, point bar
`components/TourDetail.tsx`	Passes the tour's scan points to the viewer
`components/AssetPanel.tsx`	Zip uploads, 360 badge, placed note
`components/AppHeader.tsx`	Tours and Admin portal links
`components/AdminShell.tsx` (new)	Portal guard, side menu, page title
`components/ScanPointBuilder.tsx` (new)	Six-image builder
`components/PlacementEditor.tsx` (new)	Placement table and map
`components/ActivityList.tsx` (new)	Activity entries
`app/admin/...` (new)	Layout and five pages

Also `README.md` (scan points, navigation, admin portal) and `.gitignore` (local samples).

11. How it was checked

- 107 backend tests pass on an isolated test database (65 at the end of the committed part).
- Web app: TypeScript, ESLint and a full production build pass.
- In the browser: every admin page loaded with no console errors; a scan point was built from six images with a position and heading, uploaded, processed to Ready with its placement saved, then the test tour was deleted from the portal.
- Navigation checked against Matterport's own player at the same three points.
- Not clicked in the browser: changing a user's role, because the automated browser was not allowed to grant admin rights. The API tests cover it; try it once yourself.

12. Data cleanup

All the earlier sample tours were deleted on request. One tour remains: **Alder Hey Children's Hospital / Atrium (Matterport scan points)**, with three placed scan points (3 zips plus 18 faces in storage). It is a draft, so only admins see it until it is published.

13. Running it locally

```
cd tourvault
docker compose up -d --build        # API, Postgres, S3 storage, legacy PHP, nginx

cd web
cp .env.example .env.local           # NEXT_PUBLIC_API_URL=http://localhost:8001
npm install
npm run dev                          # http://localhost:3000
```

- API docs: http://127.0.0.1:8001/docs. Admin portal: http://localhost:3000/admin.
- New accounts are viewers. Make the first admin in the database (README has the command); after that, use the Users page.
- Backend tests: .venv/Scripts/python -m pytest -q from the tourvault folder.

14. Limits and what to do next

- Nothing from parts 4 to 6 is committed yet.
- Placement is typed in by hand. A next step would be dragging points on the map.
- Rings use straight-line distance. Matterport also knows which points can see each other; TourVault would need that as extra data.
- No image resizing: very large faces are served as they are. Smaller copies for phones would load faster.
- The admin guard in the browser is cosmetic by design; the server is the authority.

15. Questions you may be asked

Why did email-only login work?

Registration let an empty password through. The stored hash was of an empty string, so logging in with nothing matched. The fix is at both ends: registration demands a real password and login refuses an empty one.

Why 72 bytes?

bcrypt only uses the first 72 bytes. Longer passwords would be cut silently, so they are refused at registration and treated as a wrong password at login.

Why hash a dummy password for unknown emails?

So a wrong email takes as long as a wrong password. Otherwise response time tells an attacker which emails exist.

Why not convert Matterport scans to GLB?

A scan point is six photos around one spot, not a mesh. Showing them as a cube is what Matterport itself does, keeps full photo quality, and needs no conversion step that could lose or distort anything.

How do you stop a malicious zip?

Limits on entry count and size, no paths that climb out of the folder, every image opened and checked by Pillow, and point.json size and number checks. Faces are read in memory, never extracted to disk.

How do points know where they are?

Each has a position and heading, from point.json or set by an admin. The viewer draws a ring for each nearby point at floor level and moves there on click.

What stops two admins demoting each other?

The role change and delete run inside a transaction that locks the users table first, so the second admin sees the result of the first change and is refused if it would leave no admin.

Why build the zip in the browser?

So the server keeps one upload format and one set of checks. The builder is just a friendlier way to make that zip.

Why can't a user with activity be deleted?

The audit log records who made each change. Deleting the user would break that record, so they are made a viewer instead.

Why does the browser get presigned links?

Files go straight from storage to the browser, so large models and faces never pass through the API, and each link expires after 10 minutes.